Thành viên	Nội dung	Sản phẩm
Nguyễn Văn Hiếu	Dự kiến thiết kế phần cứng theo blocks	Block diagram, mô tả các module, luồng dữ liệu
Nguyễn Đình Tuyển	Phân tích cách thuật toán hoạt động, flowchart, pseudo-code, ví dụ tính toán	Nguyên lý hoạt động và ví dụ
Vũ Nguyễn Đăng Nhật	Lý thuyết thuật toán Viterbi, trellis, metric, survivor path	Cơ sở lý thuyết, hình minh họa

1. Lý thuyết thuật toán

1.1. Giới thiệu thuật toán

Khi dữ liệu được truyền qua một kênh có nhiễu, một số bit tại bên nhận có thể bị lật từ 0 thành 1 hoặc ngược lại. Để hỗ trợ khôi phục dữ liệu bị sai lệch, bên gửi có thể sử dụng mã hóa chập (convolutional coding), trong đó các bit dư được tạo ra theo một quy tắc xác định trước.

Cách đơn giản nhất là thử mọi chuỗi bit ban đầu có thể có, mã hóa lại từng chuỗi, so với chuỗi nhận được và giữ chuỗi giống nhất. Tin nhắn dài N bit có $O(2^N)$ chuỗi cần thử, nên cách này không dùng được ngay cả khi N chỉ vài chục.

Thuật toán Viterbi, do Andrew Viterbi đề xuất năm 1967, tìm đúng chuỗi giống nhất đó nhưng chỉ cần $O(2^2xN)$. Ý tưởng cốt lõi là khi hai đường đi cùng đến một trạng thái ở cùng một thời điểm, chỉ cần giữ đường có chi phí thấp hơn. Đường còn lại không thể nằm trong lời giải tốt nhất, vì phần đường đi tiếp sau đó là như nhau với cả hai.

1.2. Ứng dụng của thuật toán

Bảng 1. Một số ứng dụng của thuật toán Viterbi

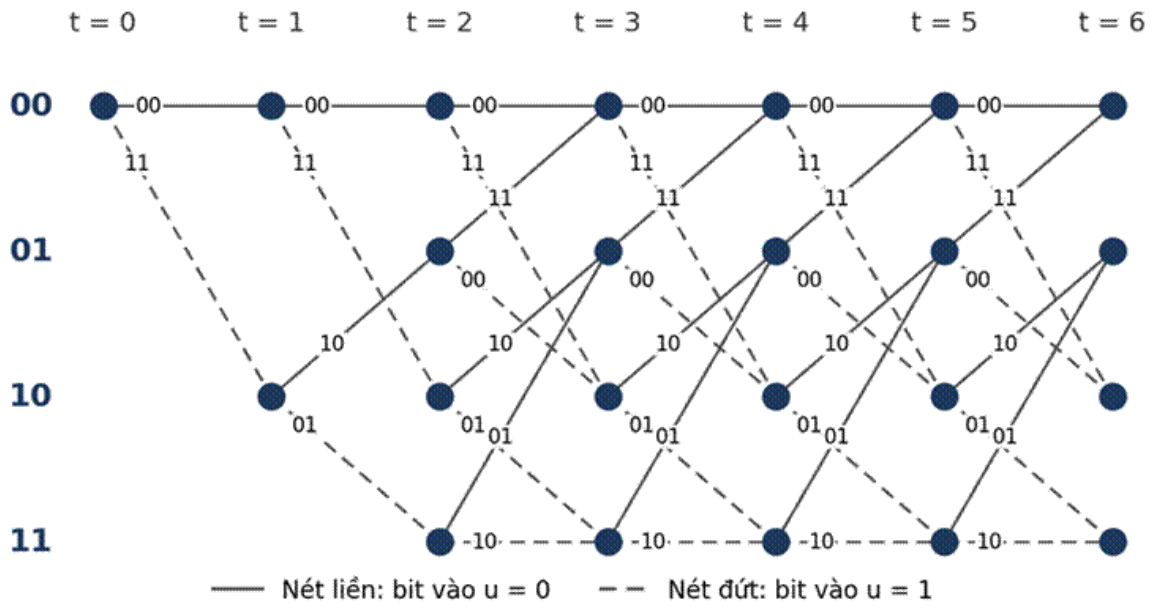
Lĩnh vực	Mục đích sử dụng
Nhận dạng giọng nói	Thuật toán giúp biến đổi âm thanh thành văn bản
Vi sinh học	Thuật toán giúp xác định DNA và mã di truyền
Xử lý ngôn ngữ tự nhiên	Gắn thẻ từ loại và nhận dạng thực thể
Truyền thông	Sửa lỗi bằng cách giải mã các mã chập

1.3. Các khái niệm

1.3.1. Sơ đồ trellis

Trellis là sơ đồ trải các state theo thời gian. Mỗi cột ứng với một thời điểm, mỗi hàng ứng với một state, và mỗi nhánh nối một state ở thời điểm t với một state ở thời điểm t+1. Trong hình dưới đây, nhánh nét liền ứng với bit vào 0, nhánh nét đứt ứng với bit vào 1, và cặp số ghi trên nhánh là cặp bit ra. Bộ mã hóa luôn bắt đầu ở state 00, nên ở những bước đầu chỉ một số nút đến được.

Mỗi chuỗi bit vào cụ thể ứng với đúng một đường đi từ trái sang phải trên trellis. Vì vậy bài toán giải mã trở thành bài toán chọn một đường đi trên trellis, sao cho cặp bit ra dọc đường đi giống nhất với dữ liệu nhận được.



1.3.2. Trạng thái (State)

State là nội dung bộ nhớ của bộ mã hóa, gồm K−1 bit vào gần nhất. Với K = 3 có $2^{K-1} = 4$ state: 00, 01, 10 và 11. Quy ước viết state là s_1s_0, bit bên trái là bit vào gần nhất.

State quyết định hai điều: bit ra sẽ là gì ứng với mỗi bit vào, và state tiếp theo có thể là gì. Từ mỗi state chỉ có hai state kế tiếp, ứng với bit vào 0 hoặc 1.

1.3.3. Chi phí nhánh (Branch Metric)

Branch metric là chi phí của một nhánh, đo độ khác nhau giữa cặp bit ra gắn với nhánh và cặp bit thực tế nhận được ở thời điểm đó. Vì mỗi cặp có 2 bit nên BM chỉ nhận giá trị 0, 1 hoặc 2.

Bảng 2. Branch metric

	00	**01**	**10**	**11**
00	0	1	1	2
01	1	0	2	1
10	1	2	0	1
11	2	1	1	0

1.3.4. Chi phí đường đi (Path Metric)

Path metric (PM[s,t]) là giá trị đại diện cho tổng lỗi tích lũy ở state s và thời gian t.

Từ định nghĩa này ta hoàn toàn có thể suy ra, PM[s,t] là tổng BM nhỏ nhất các đường hợp lệ đến trạng thái s ở thời điểm t; ban đầu PM của `00` bằng 0, các trạng thái khác bằng ∞, biểu diễn bằng 31. Đồng thời, đường đi có PM nhỏ nhất tại mỗi thời điểm tương ứng với chuỗi thông tin có xác suất được truyền đi cao nhất.

1.3.5. Đường sống sót (Survivor Path)

Mỗi state có hai nhánh đi vào. Sau khi so sánh, chỉ nhánh cho Path metric nhỏ hơn được giữ lại. Đường tốt nhất từ đầu trellis đến state đó, đi qua nhánh được giữ, gọi là survivor path.

1.3.6. Truy vết (Traceback)

Traceback là bước cuối cùng. Từ state kết thúc, lần ngược theo thông tin survivor qua từng thời điểm cho đến state ban đầu, rồi đọc bit vào của các nhánh đã đi qua theo chiều xuôi. Vì bit vào của một nhánh bằng bit đầu tiên của state đích (mục 1.3.2), có thể đọc bit trực tiếp từ dãy state tìm được. Với tin nhắn có độ dài hữu hạn, traceback chỉ thực hiện sau khi đã tính xong Path metric cho toàn bộ chuỗi.

1.3.7. Cộng, so sánh và lựa chọn (ACS)

ACS là phép tính lõi, lặp lại ở mọi state tại mọi thời điểm. Mỗi lần gồm các bước sau:

Bước 1: Add: với mỗi nhánh trong hai nhánh đi vào state, cộng Path metric của state cha với Branch metric của nhánh.

Bước 2: Compare: so sánh hai tổng vừa có.

Bước 3: Select: giữ tổng nhỏ hơn làm PM mới của state và ghi lại state cha tương ứng.

Khi hai tổng bằng nhau, chọn nhánh nào cũng được, nhưng cần cố định một quy ước chọn để kết quả lặp lại được.

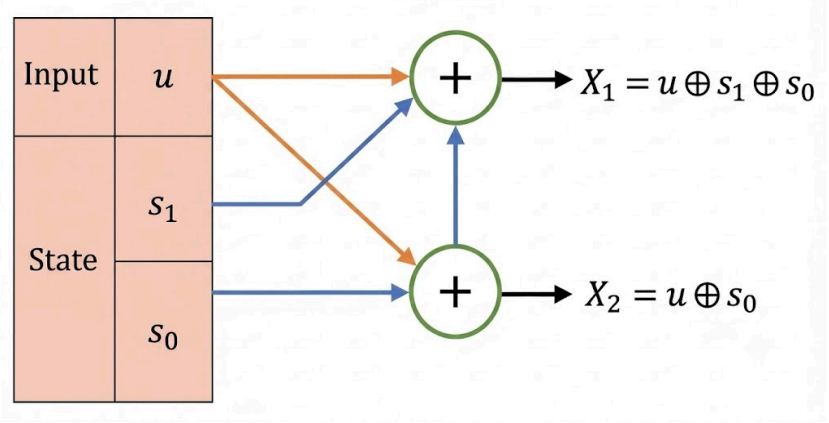
1.4. Mô tả dữ liệu vào và ra

Bảng 3. Input và output ở mức thuật toán

Mục	Bộ mã hóa (bên gửi)	Bộ giải mã Viterbi (cần thiết kế)
Input	Chuỗi bit tin nhắn, mỗi lần vào 1 bit	Chuỗi cặp bit nhận được, mỗi thời điểm một cặp
Output	Chuỗi cặp bit mã, mỗi bit vào cho ra 2 bit	Chuỗi bit tin nhắn ước lượng, 1 bit cho mỗi cặp nhận vào
Tham số cố định	K = 3, g_1 = 111, g_2 = 101	K = 3, g_1 = 111, g_2 = 101
Dữ liệu trong ví dụ	Vào: 0 1 1 0 0 1 0 0. Ra: 00 11 01 01 11 11 10 11	Vào: 00 11 11 01 10 11 10 11. Ra: 0 1 1 0 0 1 0 0

1.5. Ví dụ minh họa

Ví dụ dùng bộ mã hóa đã giới thiệu với tin nhắn 8 bit 01100100. K = 3, g1 = 111, g2 = 101 (g là đa thức sinh).



Bảng 4. Bảng giá trị chân lý

u	s_1	s_0	X_1	X_2
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	1
1	0	1	0	0
1	1	0	0	1
1	1	1	1	0

Từ bảng giá trị chân lý này, ta có thể suy ra như sau:

- Bit vào: u
- State trước: s_1s_0
- Cặp bit ra: X_1X_2 (với X_1 = u XOR s_1 XOR s_0, X_2 = u XOR s_0)
- State sau được cập nhật theo quy tắc: $s_1s_0 \rightarrow us_1$

Và trạng thái us_1 của thời điểm t trở thành trạng thái trước s_1s_0 của thời điểm t+1.

1.5.1. Mã hóa bên gửi

Bảng 5. Quá trình mã hóa tin nhắn 01100100

t	Bit vào u	State trước	Cặp bit ra	State sau
1	**0**	**00**	**00**	**00**
2	**1**	**00**	**11**	**10**

3	**1**	**10**	**01**	**11**
4	**0**	**11**	**01**	**01**
5	**0**	**01**	**11**	**00**
6	**1**	**00**	**11**	**10**
7	**0**	**10**	**10**	**01**
8	**0**	**01**	**11**	**00**

Chuỗi gửi đi là 00 11 01 01 11 11 10 11.

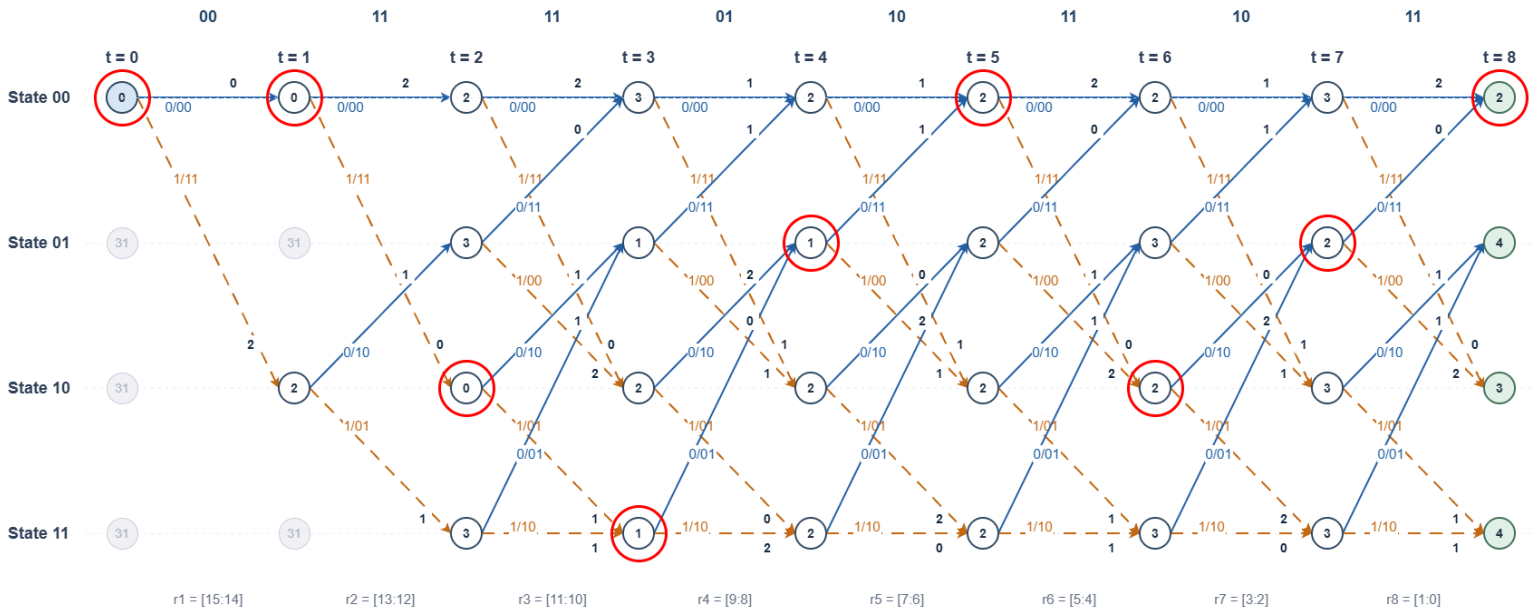
1.5.2. Kênh gây nhiễu

Giả sử kênh gây 2 lỗi: ở t = 3 cặp 01 bị thành 11, ở t = 5 cặp 11 bị thành 10. Chuỗi nhận được là 00 11 11 01 10 11 10 11. Bên nhận không biết lỗi nằm ở đâu.

1.5.3. Giải mã Viterbi

Ở mỗi thời điểm, bộ giải mã tính Branch metric của từng nhánh theo Bảng 2 và thực hiện ACS cho 4 state.

Hình 2. Quá trình giải mã



Bảng 6. Chi tiết phép ACS tại t = 3

State đích	State cha	Cặp bit ra của nhánh	PM cũ	BM	Tổng	Kết quả
00	00	00	2	2	4	Loại
00	**01**	**11**	**3**	**0**	**3**	**Giữ**

01	**10**	**10**	**0**	**1**	**1**	**Giữ**
01	11	01	3	1	4	Loại
10	**00**	**11**	**2**	**0**	**2**	**Giữ**
10	01	00	3	2	5	Loại
11	**10**	**01**	**0**	**1**	**1**	**Giữ**
11	11	10	3	1	4	Loại

Các dòng tô màu là nhánh được giữ, trở thành survivor của state đích.

1.5.4. Truy vết

Traceback bắt đầu từ state 00 (chọn state 00 vì PM = 2 nhỏ nhất duy nhất) ở t = 8 (PM = 2) và lần ngược theo cột state cha:

t = 8: state 00, cha là 01 (t = 7)

t = 7: state 01, cha là 10 (t = 6)

t = 6: state 10, cha là 00 (t = 5)

t = 5: state 00, cha là 01 (t = 4)

t = 4: state 01, cha là 11 (t = 3)

t = 3: state 11, cha là 10 (t = 2)

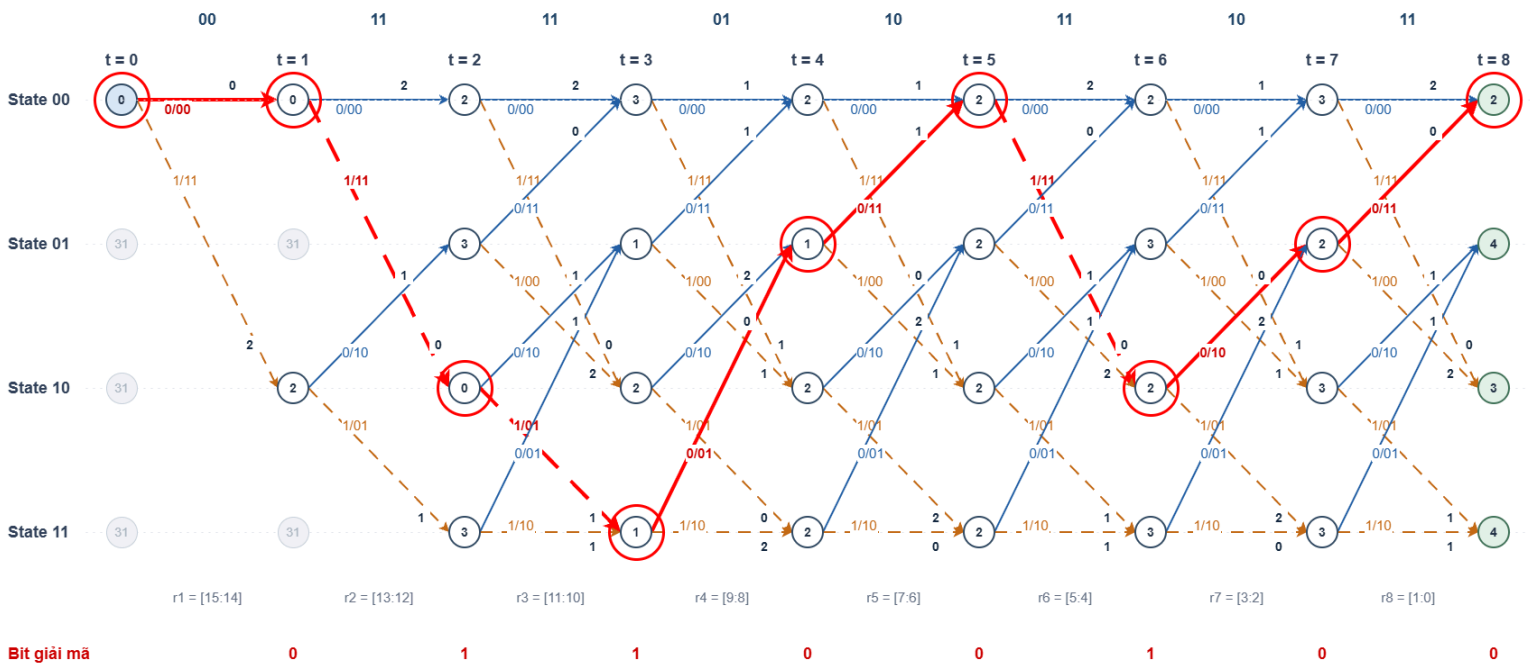
t = 2: state 10, cha là 00 (t = 1)

t = 1: state 00, cha là 00 (t = 0)

Viết theo chiều xuôi, dãy state là 00, 00, 10, 11, 01, 00, 10, 01, 00. Bit vào ở mỗi bước bằng bit đầu tiên của state đích, nên đọc được 01100100. Đúng tin nhắn ban đầu dù kênh đã gây 2 lỗi.

Chú ý thêm:

Tại thời điểm kết thúc, chọn trạng thái có PM nhỏ nhất để bắt đầu truy vết. Nếu nhiều trạng thái cùng có PM nhỏ nhất, ưu tiên theo thứ tự 00, 01, 10, 11 để kết quả lựa chọn luôn xác định. Thiết kế không bắt buộc kết thúc tại state 00.



2. Nguyên lý hoạt động và luồng xử lý

2.1. Các bước xử lý của thuật toán

Thuật toán giải mã Viterbi (với quyết định cứng - Hard-decision Viterbi Decoding) hoạt động dựa trên phương pháp quy hoạch động (Dynamic Programming), tìm đường đi có tổng khoảng cách Hamming tích lũy nhỏ nhất trên lưới Trellis. Quá trình giải mã được chia thành 4 giai đoạn tuần tự:

2.1.1. Khởi tạo

- **Thời điểm** $t = 0$**:** Bộ mã hóa luôn bắt đầu từ trạng thái reset ban đầu là $S_0(00)$. Do đó:

$$PM(00) = 0$$

$$PM(01) = PM(10) = PM(11) = \infty \quad (trong\ phần\ cứng\ gán\ bằng\ một\ giá\ trị\ cực\ đại\ MAX_VAL\ )$$

- Việc gán ∞ đảm bảo các đường đi xuất phát từ những trạng thái không hợp lệ ở thời điểm bắt đầu sẽ bị loại bỏ hoàn toàn trong các bước so sánh tiếp theo.

2.1.2. Tính chi phí nhánh (BM)

- Ở mỗi chu kỳ thời gian t, bộ giải mã nhận một cặp bit từ kênh truyền: $r(t) = (r_1, r_2)$.
- Với mỗi nhánh chuyển trạng thái từ trạng thái cũ S_{prev} sang trạng thái mới S_{next}, cặp bit phát lý thuyết tương ứng là $x(t) = (x_1, x_2)$ (tra theo bảng chân lý mã hóa).

- Branch Metric là khoảng cách Hamming giữa cặp bit nhận được và cặp bit lý thuyết của nhánh:

$$BM = \left(r_1 \oplus x_1\right) + \left(r_2 \oplus x_2\right) \in \{0, 1, 2\}$$

2.1.3. Cộng, so sánh và lựa chọn (ACS)

Với mỗi trạng thái đích $S_d \in \{00, 01, 10, 11\}$ ở thời điểm t, luôn có đúng hai trạng thái cha S_a và S_b ở thời điểm $t - 1$ có thể chuyển tới:

1. **Add (Cộng):** Tính chi phí ứng viên tích lũy từ hai nhánh:

$$Candidate_A = PM_{t-1}\left(S_a\right) + BM\left(S_a \to S_d\right)$$

$$Candidate_B = PM_{t-1}\left(S_b\right) + BM\left(S_b \to S_d\right)$$

2. **Compare (So sánh):** So sánh giá trị giữa $Candidate_A$ và $Candidate_B$.

3. **Select (Lựa chọn & Cập nhật):**

$$PM_t\left(S_d\right) = min\left(Candidate_A, Candidate_B\right)$$

   - Lưu bit quyết định đường sống sót (**Decision Bit / Survivor Bit**):

$$Decision\left(t, S_d\right) = \{0 \text{ nếu chọn nhánh từ } S_a,\ 1 \text{ nếu chọn nhánh từ } S_b\}$$

(Quy ước: Nếu $Candidate_A = Candidate_B$, ưu tiên chọn nhánh từ S_a để đảm bảo tính xác định cho phần cứng).

2.1.4. Lưu quyết định survivor và truy vết

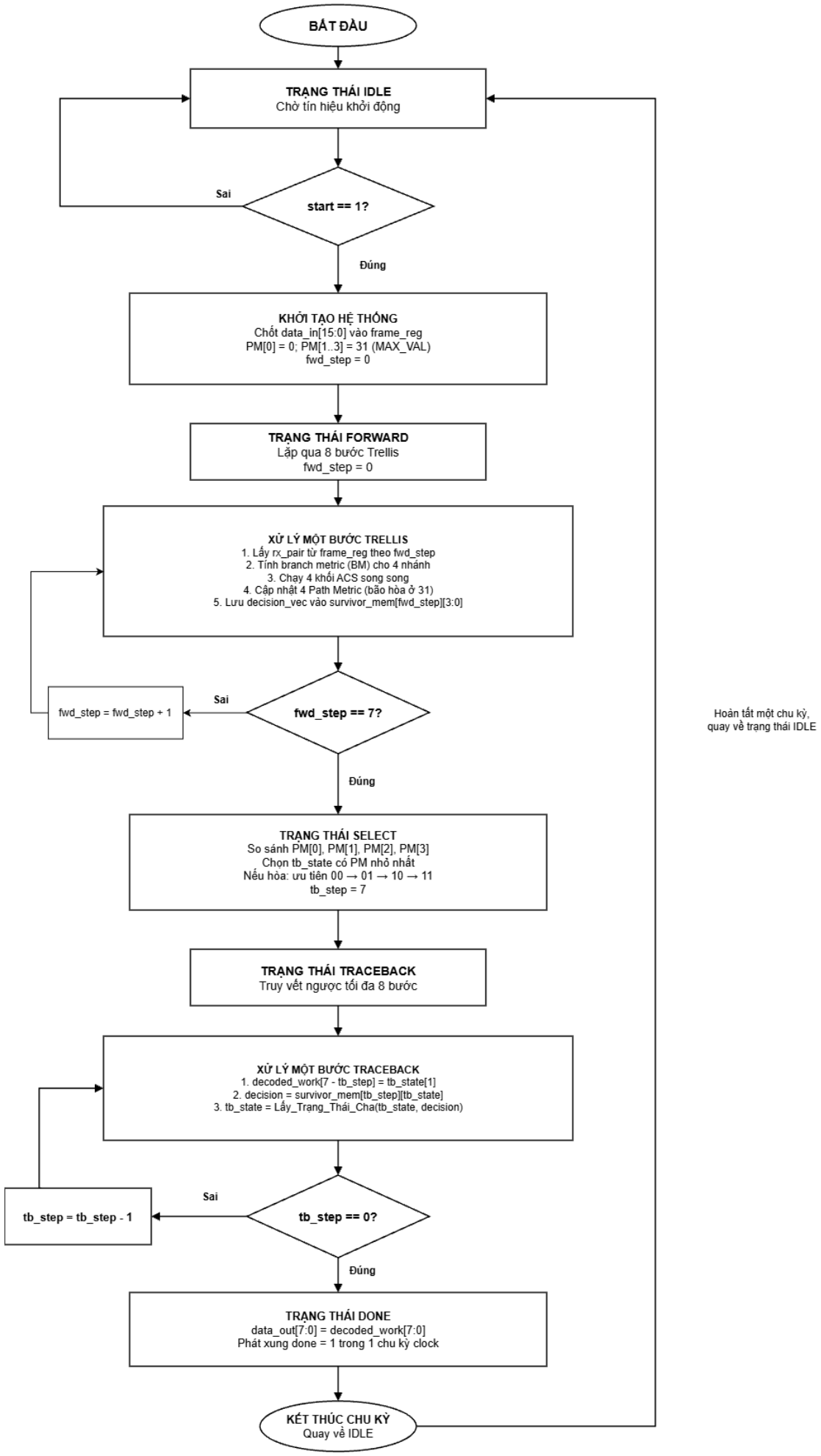
- **Ghi nhận lịch sử:** Ở mỗi bước thời gian t, lưu lại $2^{K-1} = 4$ bit quyết định của 4 trạng thái vào bộ nhớ lưu vết (Survivor Memory matrix kích thước $4 \times N$).
- **Lần ngược (Traceback):**
  - Tại bước cuối cùng $t = N$, tìm trạng thái có PM_N nhỏ nhất. Nếu nhiều trạng thái cùng PM nhỏ nhất, ưu tiên theo thứ tự `00 → 01 → 10 → 11`.
  - Dựa vào Survivor Bit đã lưu tại trạng thái đó, xác định ngược lại trạng thái cha ở bước $t - 1$.
  - Theo quy ước chuyển trạng thái s1s0 → us1, bit thông tin gửi đi u chính là bit có trọng số cao nhất (MSB - bit bên trái) của trạng thái đích:

$$\hat{u}_t = State_{dest}[1]$$

  - Quá trình truy vết thực hiện từ `t = N` về `t = 1`. Với `N = 8`, bit giải mã tại bước `t` được ghi vào `decoded_work[8 - t]`.

2.2. Lưu đồ thuật toán

LƯU ĐỒ THUẬT TOÁN GIẢI MÃ VITERBI



2.3. Mã giả

```
THUẬT TOÁN: Viterbi_Decoder_Hardware_Level
ĐẦU VÀO:
  - clk, rst_n, start: Các tín hiệu điều khiển 1-bit
  - data_in[15:0]: Khung 16-bit dữ liệu mã hóa từ kênh truyền
ĐẦU RA:
  - data_out[7:0]: 8-bit dữ liệu gốc được khôi phục
  - done: Cờ báo hoàn thành (1 chu kỳ)

HẰNG SỐ:
  - MAX_VAL = 31                  // Giá trị cực đại của thanh ghi PM 5-bit
  - TOTAL_STEPS = 8               // 8 bước xử lý trellis

BẮT ĐẦU
  KHI rst_n == 0 THÌ: // Reset ưu tiên; survivor_mem không reset, chỉ được đọc sau khi đã ghi mới đủ 8 hàng:
frame_load=frame_init=metric_en=survivor_we=select_en=tb_en=out_en=0
    state = IDLE
    done = 0
    data_out = 0

    frame_reg = 0
    decoded_work = 0
    tb_state = 0
    fwd_step = 0
    tb_step = 0

    PM[0..3] = MAX_VAL

  KHI CẠNH LÊN clk và rst_n == 1: // Enable tổ hợp theo trạng thái trước cạnh: IDLE + start -> frame_load, frame_init; FORWARD -> metric_en, survivor_we; SELECT -> select_en; TRACEBACK -> tb_en; DONE -> out_en; enable khác = 0.
    TRƯỜNG HỢP (state):

      // GIAI ĐOẠN 1: CHỜ LỆNH BẮT ĐẦU
      IDLE:
        done = 0
        NẾU start == 1 THÌ:
          frame_reg = data_in[15:0]    // Chốt 16-bit vào frame_reg
          PM[0] = 0                   // Trạng thái 00 xuất phát từ 0
          PM[1] = MAX_VAL
          PM[2] = MAX_VAL
          PM[3] = MAX_VAL
          fwd_step = 0
          state = FORWARD
        HẾT NẾU

      // GIAI ĐOẠN 2: TÍNH TIẾN METRIC & ACS (8 CHU KỲ CLOCK)
```

```
      FORWARD:
        // Lấy từng cặp bit theo thứ tự từ trái sang phải
        rx_pair = frame_reg[15 - 2*fwd_step : 14 - 2*fwd_step]

        // Thực hiện 4 khối ACS song song cho 4 trạng thái đích (next_s):
        CHO next_s = 0 ĐẾN 3 THỰC HIỆN:
          // Với trạng thái đích next_s = {u, s1}, trạng thái cha s1s0 = {next_s[0],
bit_s0}
          cha_A = Lấy_Trạng_Thái_Cha(next_s, bit_s0 = 0)   // tức là {next_s[0], 1'b0}
          cha_B = Lấy_Trạng_Thái_Cha(next_s, bit_s0 = 1)   // tức là {next_s[0], 1'b1}

          bm_A = Hamming(rx_pair, Bit_Ra_Lý_Thuyết(cha_A -> next_s))
          bm_B = Hamming(rx_pair, Bit_Ra_Lý_Thuyết(cha_B -> next_s))

          // PM + BM được cộng bằng 6 bit không dấu,
          // sau đó bão hòa tại MAX_VAL = 31.
          cand_A = (PM[cha_A] == MAX_VAL) ? MAX_VAL : min(31, PM[cha_A] + bm_A)
          cand_B = (PM[cha_B] == MAX_VAL) ? MAX_VAL : min(31, PM[cha_B] + bm_B)

          // Add - Compare - Select (Ưu tiên nhánh A nếu hòa):
          NẾU cand_A <= cand_B THÌ:
            Next_PM[next_s] = cand_A
            survivor_mem[fwd_step][next_s] = 0   // Ghi bit quyết định chọn cha A
          NGƯỢC LẠI:
            Next_PM[next_s] = cand_B
            survivor_mem[fwd_step][next_s] = 1   // Ghi bit quyết định chọn cha B
          HẾT NẾU
        HẾT VÒNG LẶP

        PM[0..3] = Next_PM[0..3]     // Cập nhật đồng bộ tại sườn clock
        // Kiểm tra bước 7 trước khi tăng bộ đếm 3 bit

        NẾU fwd_step == TOTAL_STEPS - 1 THÌ:
          state = SELECT
        NGƯỢC LẠI:
          fwd_step = fwd_step + 1

        HẾT NẾU

      // GIAI ĐOẠN 3: CHỌN TRẠNG THÁI BẮT ĐẦU TRUY VẾT (1 CLOCK)
      SELECT:
        // Chọn trạng thái có PM nhỏ nhất. Nếu bằng nhau, ưu tiên 00 -> 01 -> 10 -> 11
        tb_state = Chọn_Trạng_Thái_Min(PM[0], PM[1], PM[2], PM[3])
        tb_step = TOTAL_STEPS - 1    // Bắt đầu lần vết từ bước 7 (bước thứ 8)
        decoded_work = 0
        state = TRACEBACK

      // GIAI ĐOẠN 4: LẦN VẾT NGƯỢC (8 CHU KỲ CLOCK)
      TRACEBACK:
        // Bit giải mã là bit MSB của trạng thái đích hiện tại:
        // Bước 8 (tb_step=7) ghi vào decoded_work[0], bước 1 (tb_step=0) ghi vào [7]
```

```
        decoded_work[7 - tb_step] = tb_state[1]

        // Tra cứu quyết định rẽ nhánh từ survivor_mem:
        decision = survivor_mem[tb_step][tb_state]
        tb_state = Lấy_Trạng_Thái_Cha(tb_state, decision) // {tb_state[0], decision}


        NẾU tb_step == 0 THÌ:
          state = DONE
        NGƯỢC LẠI:
          tb_step = tb_step - 1
        HẾT NẾU

      // GIAI ĐOẠN 5: XUẤT KẾT QUẢ VÀ BÁO HOÀN THÀNH (1 CLOCK)
      DONE:
        data_out[7:0] = decoded_work[7:0]
        done = 1
        state = IDLE

      TRƯỜNG HỢP KHÁC:
        // Tắt toàn bộ tín hiệu điều khiển
        // Logic tổ hợp
        frame_load  = 0  // Không chốt khung data_in mới
        frame_init  = 0  // Không khởi tạo lại PM
        metric_en   = 0  // Không cập nhật các thanh ghi PM
        survivor_we = 0  // Không ghi bộ nhớ survivor_mem
        select_en   = 0  // Không chọn tb_state hoặc xóa decoded_work
        tb_en       = 0  // Không thực hiện bước truy vết
        out_en      = 0  // Không chuyển decoded_work sang data_out

        // Hủy khung đang xử lý và trở về chờ
        // Theo cạnh lên clk
        state = IDLE
        done = 0
        fwd_step = 0
        tb_step = 0

        // Giữ nguyên:
        // data_out, frame_reg, PM[0..3],
        // decoded_work, tb_state và survivor_mem
  HẾT CHU KỲ CLOCK
KẾT THÚC
```

2.3.1. Bảng chú giải các biến và tín hiệu

Tên biến / Tín hiệu	Độ rộng	Vị trí / Thuộc khối	Ý nghĩa chức năng
clk, rst_n	1 bit	Toàn hệ thống	Clock sườn lên và Reset tích cực mức thấp.
start	1 bit	Controller	Xung kích hoạt bắt đầu quá trình giải mã khung mới.
data_in[15:0]	16 bit	Input Buffer	Khung dữ liệu vào gồm 8 cặp bit mã hóa.
data_out[7:0]	8 bit	Traceback / Output	Kết quả 8 bit dữ liệu giải mã hoàn chỉnh.
done	1 bit	Controller	Báo hoàn thành giải mã trong 1 chu kỳ clock.
PM[0..3]	5 bit × 4	Metric Update	4 thanh ghi lưu Path Metric của 4 trạng thái (00, 01, 10, 11).
survivor_mem[0..7][0..3]	32 bit	Survivor Memory	Bộ nhớ ma trận lưu các decision bit qua 8 bước trellis.
decoded_work[7:0]	8 bit	Traceback	Thanh ghi chốt kết quả giải mã tạm thời khi lần vết lùi.
tb_state	2 bit	Traceback	Trạng thái đang được xét trong quá trình lần vết.
MAX_VAL	5 bit (5'd31)	Hằng số	Giá trị khởi tạo cực đại để chống tràn số cho thanh ghi PM.

2.4. Đối chiếu ví dụ tính toán

1. Bảng chuyển trạng thái & Cặp bit phát lý thuyết

Từ bảng chân lý mục 1.5 (X1 = u XOR s1 XOR s0, X2 = u XOR s0):

- Nhánh đi vào trạng thái đích S_next = us1 gồm 2 trạng thái cha S_prev = s1s0 (Cha A ứng với s0 = 0, Cha B ứng với s0 = 1):
    - **Đích 00 (u = 0, s1 = 0):**
        - Từ cha A = 00 (s0 = 0): Cặp bit phát = 00
        - Từ cha B = 01 (s0 = 1): Cặp bit phát = 11
    - **Đích 01 (u = 0, s1 = 1):**
        - Từ cha A = 10 (s0 = 0): Cặp bit phát = 10
        - Từ cha B = 11 (s0 = 1): Cặp bit phát = 01

- **Đích 10 (u = 1, s1 = 0):**
  - Từ cha A = 00 (s0 = 0): Cặp bit phát = 11
  - Từ cha B = 01 (s0 = 1): Cặp bit phát = 00
- **Đích 11 (u = 1, s1 = 1):**
  - Từ cha A = 10 (s0 = 0): Cặp bit phát = 01
  - Từ cha B = 11 (s0 = 1): Cặp bit phát = 10

2.5. Diễn giải quá trình ACS qua 8 bước ($t = 1 \to 8$)

- Cặp bit nhận được từng thời điểm (tương ứng bản tin gốc 01100100 trong mục 1.5): R = [00, 11, 11, 01, 10, 11, 10, 11] (bị lật bit tại t = 3 từ 01 thành 11 và tại t = 5 từ 11 thành 10).

t	r(t)	PM00	PM01	PM10	PM11	Cha sống sót (00, 01, 10, 11)	Survivor Bit
t = 0	Khởi tạo	0	∞	∞	∞	—	—
t = 1	00	0	∞	2	∞	00 / — / 00 / —	0 / — / 0 / —
t = 2	11	2	3	0	3	00 / 10 / 00 / 10	0 / 0 / 0 / 0
t = 3 (lỗi)	11	3	1	2	1	01 / 10 / 00 / 10	1 / 0 / 0 / 0
t = 4	01	2	1	4	1	01 / 11 / 00 / 10	1 / 1 / 0 / 0
t = 5 (lỗi)	10	2	3	2	3	01 / 11 / 01 / 11	1 / 1 / 1 / 1
t = 6	11	4	3	2	3	00 / 10 / 00 / 10	0 / 0 / 0 / 0
t = 7	10	4	2	4	4	01 / 10 / 01 / 10	1 / 0 / 1 / 0
t = 8	11	2	5	4	5	01 / 10 / 00 / 10	1 / 0 / 0 / 0

2.6. Quá trình truy vết và khôi phục bản tin

- Tại t = 8: Trạng thái có metric nhỏ nhất là 00 với PM = 2. Chọn trạng thái 00 để bắt đầu truy vết.
- Bắt đầu lần vết ngược từ trạng thái 00 tại t = 8 về t = 1:

1. t = 8: Ở trạng thái 00, decision = 1 → cha sống sót là 01. Bit khôi phục: u_8 = MSB(00) = 0.
2. t = 7: Ở trạng thái 01, decision = 0 → cha sống sót là 10. Bit khôi phục: u_7 = MSB(01) = 0.
3. t = 6: Ở trạng thái 10, decision = 0 → cha sống sót là 00. Bit khôi phục: u_6 = MSB(10) = 1.
4. t = 5: Ở trạng thái 00, decision = 1 → cha sống sót là 01. Bit khôi phục: u_5 = MSB(00) = 0.

5. t = 4: Ở trạng thái 01, decision = 1 → cha sống sót là 11. Bit khôi phục: u_4 = MSB(01) = 0.
6. t = 3: Ở trạng thái 11, decision = 0 → cha sống sót là 10. Bit khôi phục: u_3 = MSB(11) = 1.
7. t = 2: Ở trạng thái 10, decision = 0 → cha sống sót là 00. Bit khôi phục: u_2 = MSB(10) = 1.
8. t = 1: Ở trạng thái 00, decision = 0 → cha sống sót là 00. Bit khôi phục: u_1 = MSB(00) = 0.

- **Kết luận:**
  - Chuỗi bit sau khi đọc xuôi (t = 1 → 8): [0, 1, 1, 0, 0, 1, 0, 0].
  - Chuỗi giải mã trùng khớp hoàn toàn với bản tin 8-bit gốc gửi đi (01100100), chứng minh thuật toán Viterbi đã phát hiện và tự động sửa thành công 2 lỗi truyền thông trên kênh.

3. Dự kiến thiết kế phần cứng

3.1. Thông số và quy ước thiết kế

- Bộ giải mã Viterbi sử dụng khoảng cách Hamming để tính chi phí nhánh. Các thông số của thiết kế gồm:

Thông số	Giá trị
Constraint length	$K = 3$
Coding rate	$R = 1/2$
Đa thức sinh	$g_1 = 111,\ g_2 = 101$
Số trạng thái	4: 00, 01, 10, 11
Dữ liệu đầu vào	16 bit mã hóa
Dữ liệu đầu ra	8 bit giải mã
Số bước xử lý trellis	8

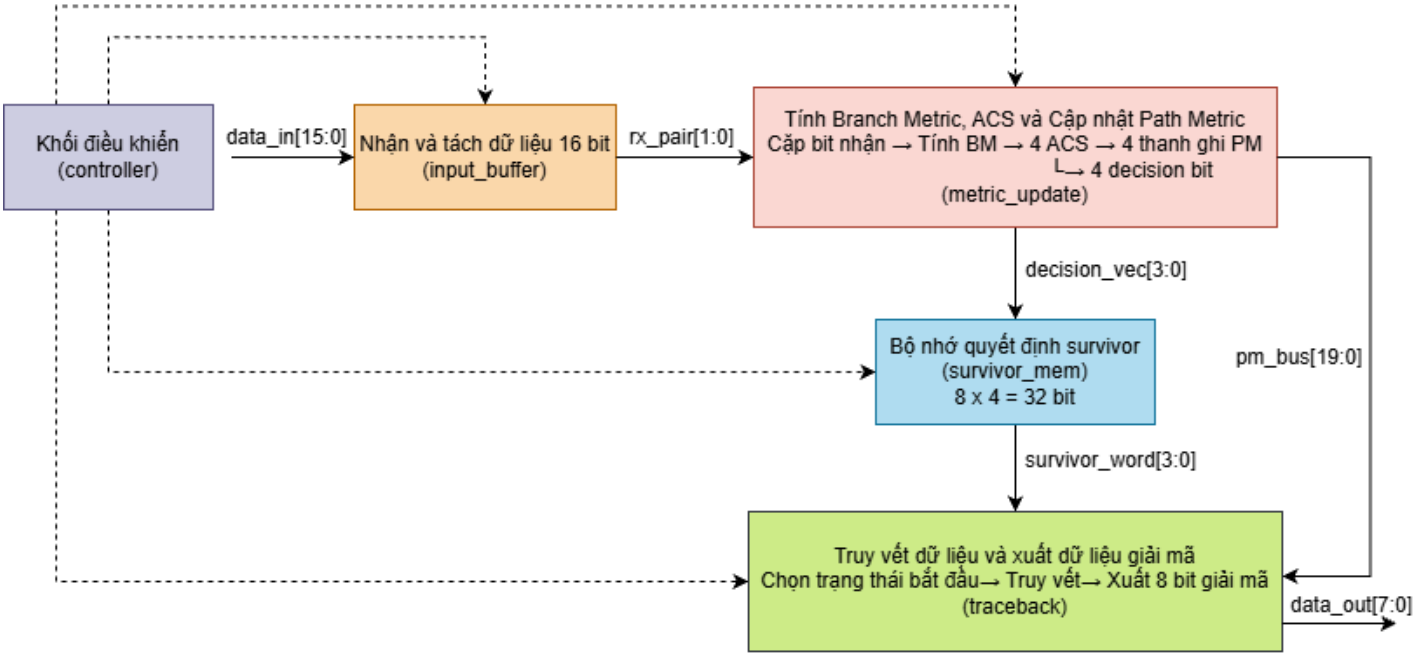
- Thiết kế xử lý từng khung độc lập, giả sử bộ mã hóa bắt đầu ở trạng thái 00. Khi nhận một khung mới, các Path Metric được khởi tạo:

$$PM(00) = 0, PM(01) = PM(10) = PM(11) = MAX_VAL\ .$$

- Dữ liệu đầu vào được xử lý theo thứ tự từ trái sang phải: cặp `data_in[15:14]` được xử lý trước, tiếp theo là [13:12], cho đến [1:0]. Bit giải mã đầu tiên tương ứng với `data_out[7]`, bit cuối tương ứng với `data_out[0]`.

- Nếu hai ứng viên trong một phép ACS có cùng metric, thiết kế ưu tiên nhánh A. Sau tám bước, trạng thái có PM nhỏ nhất được chọn để bắt đầu truy vết; nếu hòa, ưu tiên theo thứ tự 00, 01, 10, 11.

3.2. Sơ đồ khối và chức năng các khối



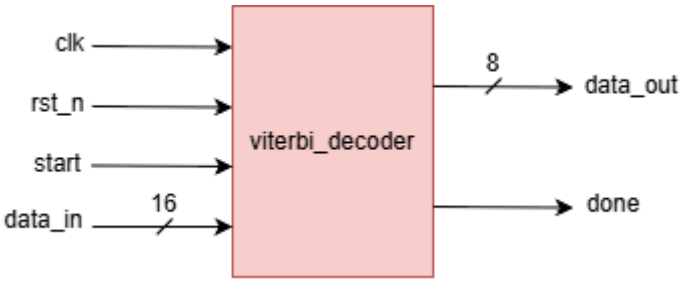
Khối	Chức năng
`controller`	Điều khiển trình tự nhận khung, khởi tạo, tính metric, ghi survivor, truy vết và báo hoàn thành.
`input_buffer`	Chốt khung đầu vào 16 bit và lần lượt cung cấp từng cặp bit cho khối tính metric.
`metric_update`	Tính Branch Metric, thực hiện bốn ACS, cập nhật bốn Path Metric và tạo 4 decision bit mỗi bước.
`survivor_mem`	Lưu các decision bit của 8 bước và cung cấp chúng cho quá trình truy vết.
`traceback`	Chọn trạng thái bắt đầu, truy ngược theo các quyết định đã lưu và sắp xếp thành 8 bit đầu ra.

- Khối `metric_update` gồm phần tính BM, bốn khối ACS và bốn thanh ghi PM. Bốn khối ACS hoạt động song song; các thanh ghi PM được cập nhật cùng lúc tại sườn lên của clock trong mỗi bước xử lý.
- Nhóm dự kiến dùng thanh ghi PM 5 bit, vì PM của một đường đi hợp lệ sau 8 bước không vượt quá 16. Giá trị `MAX_VAL` = 31 được dùng để đánh dấu trạng thái chưa

có đường đi hợp lệ. Khi PM bằng `MAX_VAL`, kết quả cộng được giữ ở 31 để tránh tràn số.

- Bộ nhớ `survivor_mem` lưu **4 decision bit ở mỗi bước**. Với 8 bước xử lý, dung lượng cần thiết là **8 × 4 = 32 bit**. Các bit này được cung cấp cho khối `traceback` khi thực hiện truy vết.

3.3. Giao diện module tổng



- Các tín hiệu vào / ra:

Tín hiệu	I/O	Độ rộng	Chức năng
`clk`	Input	1 bit	Clock hệ thống; xử lý tại cạnh lên
`rst_n`	Input	1 bit	Reset tích cực mức thấp
`start`	Input	1 bit	= 1 trong 1 chu kỳ thì bắt đầu giải mã
`data_in`	Input	16 bit	Khung dữ liệu mã hóa
`data_out`	Output	8 bit	Kết quả giải mã
`done`	Output	1 bit	= 1 là đã giải mã xong

- Khi module đang chờ, đưa dữ liệu vào `data_in` và đặt `start = 1` trong một chu kỳ clock. Tại sườn lên của clock khi `start = 1`, module lưu 16 bit đầu vào và bắt đầu giải mã.
- Trong lúc module đang giải mã, nếu có thêm yêu cầu `start`, module sẽ bỏ qua. Vì vậy, cần chờ khung hiện tại được giải mã xong rồi mới gửi khung tiếp theo.
- Khi giải mã xong, module đặt `done = 1` trong một chu kỳ clock để báo hoàn thành. Lúc này, `data_out` chứa 8 bit kết quả. Giá trị này được giữ nguyên cho đến khi có kết quả của khung tiếp theo hoặc khi reset.

3.4. Trình tự hoạt động theo clock

- Trình tự các trạng thái dự kiến: `IDLE → FORWARD → SELECT → TRACEBACK → DONE → IDLE`
- Quá trình xử lý 1 khung:

Giai đoạn	Hoạt động
Nhận khung	Khi nhận `start`, chốt dữ liệu, khởi tạo PM
FORWARD	Trong 8 clock, mỗi clock xử lý một cặp bit, cập nhật bốn PM và ghi bốn decision bit
SELECT	Chọn trạng thái bắt đầu truy vết từ 4 PM cuối
TRACEBACK	Trong 8 clock, lần ngược từ bước thứ tám về bước thứ nhất và ghi các bit vào thanh ghi kết quả tạm `decoded_work`
DONE	Chuyển kết quả hoàn chỉnh sang `data_out`, phát `done`, sau đó trở về trạng thái chờ

- Sau khi xử lý xong bước thứ tám và cập nhật đủ bốn giá trị PM, module mới chọn trạng thái để bắt đầu truy vết.
- Khi truy vết, mỗi bit giải mã được lấy từ **bit cao nhất của trạng thái hiện tại**. Module truy vết từ bước 8 về bước 1 nên bit của bước 8 được ghi vào `decoded_work[0]`, bit của bước 7 vào `decoded_work[1]`, cứ như vậy đến bit của bước 1 được ghi vào `decoded_work[7]`. Khi truy vết xong, toàn bộ 8 bit được chuyển sang `data_out` ở trạng thái DONE.
- Nhóm dự kiến đọc bộ nhớ survivor bằng logic tổ hợp. Với cách thiết kế này, module cần **18 chu kỳ clock sau khi chốt khung đầu vào**, gồm: 8 chu kỳ xử lý 8 cặp bit, 1 chu kỳ chọn trạng thái bắt đầu truy vết, 8 chu kỳ truy vết và 1 chu kỳ đưa kết quả ra `data_out`, đồng thời bật `done`.